



DESAFIO



Desenvolvimento de Sistemas Distribuídos

Aplicação de pagamento

O comércio eletrônico já é uma realidade estabelecida em nossa sociedade, onde um grande volume de negociações é realizado na internet, por meio de plataformas ou sistemas web, nos quais as lojas, comércio e indústria disponibilizam seus produtos e serviços. Para atender essa demanda cada vez maior, a tecnologia desempenha um papel crucial, pois a partir dela são disponibilizados os recursos e meios para viabilizar esse cenário.

Porém, os custos para manter grandes estruturas computacionais sempre operacionais, independentemente dos picos de volumes de transações e negociações no comércio eletrônico, têm sido um problema, por serem valores muito elevados. Dessa forma, é preciso buscar soluções que possibilitem manter a disponibilidade dessas plataformas e sistemas web para o comércio, mas buscando reduzir os custos operacionais.

Para propor uma solução, é importante conhecer todo o processo e características de funcionamento do comércio eletrônico. Sendo assim, observou-se que existem momentos específicos em que o volume de transações é maior e que, também, há partes do processamento computacional dessas transações que demandam mais recursos. Por isso, concebeu-se que, se houvesse uma possibilidade de separar as responsabilidades desses sistemas em serviços independentes, em que cada serviço pudesse ser escalonado também de forma independente, conforme a demanda, seria possível otimizar esses recursos computacionais, pois tais recursos não ficariam ociosos e seus custos seriam proporcionais ao seu uso, aumentando com as demandas por novas transações.

Diante desse contexto, a empresa CompreFácil, que possui grande parcela de suas vendas em seu e-commerce, resolveu reestruturar sua plataforma, para reduzir custos operacionais e também garantir alta disponibilidade e uma melhor experiência para seus clientes. Para atender a essa necessidade, o time de TI da empresa foi encarregado de realizar uma análise e implementar alguns serviços distribuídos para esse sistema, por meio dos conceitos da arquitetura de microsserviços, criando inicialmente dois serviços independentes: um para processar os pagamentos e outro para notificar os usuários sobre a realização da compra e confirmação de pagamento.

Para esse cenário, será preciso:

- 1) Criar um novo projeto via NodeJS para prover interfaces de comunicação (REST e AMQP) para sistema de pagamento;
- 2) Usar docker-compose para criar estruturas de banco de dados (Postgres) e sistema de mensageria (RabbitMQ);
- 3) Criar um novo projeto via NodeJS para prover interfaces de comunicação (REST e AMQP) para sistema de notificação;
- 4) Implementar fluxo de comunicação;
- 5) Receber no serviço de pagamento a solicitação de transação (o fluxo a seguir deve ser assíncrono):
 - I. sistema de pagamento armazena dados da transação com status pendente;
 - II. sistema de pagamento publica mensagem na fila para sistema de notificação informar sobre o recebimento da solicitação de transação;
 - III. sistema de notificação lê mensagem na fila e envia notificação ao usuário sobre o recebimento da solicitação de transação;
 - IV. sistema de pagamento confirma a transação, atualiza o status para sucesso;
 - V. sistema de pagamento publica mensagem na fila para sistema de notificação informar sobre a confirmação da transação;
 - VI. sistema de notificação lê mensagem na fila e envia notificação ao usuário sobre a confirmação da transação.

A seguir consulte a agenda e confira os detalhes do desafio, como: resultado esperado, forma de desenvolvimento, critérios de avaliação e forma de entrega.

AGENDA



Resultado esperado

Aplicativo funcional, com as seguintes funcionalidades:

- Serviço de pagamento sendo executado de forma independente do serviço de notificação;
- Serviço de notificação sendo executado de forma independente do serviço de pagamento;
- Comunicação assíncrona entre os serviços por meio de uma mensageria;
- Execução do fluxo de processamento para a solicitação de transação de um pagamento (descrito no desafio).

Projetos de referência:

- <https://github.com/thiagoromendes/ms-payment-service>
- <https://github.com/thiagoromendes/ms-notification-service>

Desenvolvimento

Individual. Confirmar com o professor-tutor.

Critérios de avaliação

1. Serviços independentes;
2. Comunicação com sistema de mensageria de forma assíncrona;
3. Realização do fluxo de processamento solicitado.

Forma de entrega

Entrega em ferramenta indicada pelo professor-tutor no Ambiente Virtual de Aprendizagem (AVA) das seguintes etapas:

1. Criar repositório em sistema de repositório remoto de código (p. ex.: Github);
2. Entregar o link do repositório remoto contendo o código desenvolvido e instruções de execução do projeto.